

Construção de um Compilador de COOL para Bril JSON

**Do Front-end à Geração de Código Intermediário
(IR)**

Autores: Gabriel Rodrigues e Nuno Couto

Instituição: Universidade Federal Fluminense (UFF) — Rio das Ostras

Disciplina: Compiladores

Visão Geral da Arquitetura

O pipeline clássico de compilação foi implementado de ponta a ponta:

- **Front-end:** * Analisador Léxico (`lexico.py`)
 - Analisador Sintático (`parser.py`)
 - Analisador Semântico (`semantic.py`)
- **Backend & Execução:**
 - Tradutor de AST para Bril (`cool_to_bril.py`)
 - Interpretador de Referência do Ecossistema Bril (`brili`)

Front-End: Análise Semântica

Robustecida

A análise semântica foi projetada para atuar como o núcleo de metadados do projeto:

- **Grafo de Herança:** Mapeamento completo e validação de dependências acíclicas.
- **Tabela de Símbolos:** Implementação baseada em pilhas de escopos dinâmicos para controle de visibilidade.
- **Anotação da AST:** Inferência de tipos complexos (incluindo `SELF_TYPE`), fornecendo contexto crucial para a fase de geração de código.

O Backend e a Representação Intermediária Bril

A linguagem-alvo escolhida impõe restrições educacionais desafiadoras:

- **Características do Bril:** Uma IR tipada, de baixo nível e baseada em instruções organizadas em formato JSON.
- **Estratégia de Tipagem:** O núcleo do Bril suporta nativamente apenas `int` e `bool`.
- **Solução Computacional:** Mapeamento de tipos complexos (como `String` e referências de objetos) para identificadores numéricos inteiros (`int`), simulando endereços de memória.

Desafios Técnicos Solucionados

Para que os programas COOL gerassem saídas válidas no interpretador `brili`, aplicamos engenharia de software no backend:

- **Injeção de Atributos:** Adaptação do escopo dos métodos para pré-carregar os atributos da classe e de seus ancestrais.
- **Ajuste de Entrada:** Mapeamento automático de `Main.main` (padrão COOL) para o ponto de partida global `@main` exigido pelo Bril.
- **Despachos Implícitos:** Resolução de chamadas sem receptor (ex: `out_string`) consultando a tabela semântica para encontrar o verdadeiro dono do método (`@IO.out_string`).

Infraestrutura Runtime Automatizada

Como a biblioteca padrão de COOL não existe nativamente na IR, o gerador de código injeta automaticamente funções de suporte (*stubs*):

- `IO.out_string` e `IO.in_string`: Gerenciamento de entrada e saída com alinhamento rigoroso de parâmetros (passagem explícita do ponteiro `self`).
- `String.concat`: Suporte a manipulação de strings operando sobre IDs.
- **Mapeamento de Literais**: Tradução de constantes de texto do código para inteiros correspondentes, embutindo o texto original como anotações.

Demonstração de Resultados e Validação

A pipeline integrada valida a corretude do código gerado de forma automatizada:

```
# Execução do pipeline unificado e geração da IR  
python3 -B testar_pipeline.py
```

```
# Interpretação direta do JSON resultante  
Get-Content out/final.json | brili
```